

A The HULK Programming Language

In this final part of the book, we present a straightforward and comprehensive definition of the HULK programming language, along with a set of possible extensions. This part formalizes the language we’ve been working on the entire book, and should serve as a reference for anyone wanting to build their own HULK compiler.

A.1 HULK in a nutshell

HULK (**H**avana **U**niversity **L**anguage for **K**ompilers) is a didactic, type-safe, object-oriented and incremental programming language, designed for the course Introduction to Compilers in the Computer Science major at University of Havana.

A simple “Hello World” in HULK looks like this:

```
print("Hello World");
```

In a bird’s eye view HULK is an object-oriented programming language, with simple inheritance, polymorphism, and encapsulation at the class level. Also, in HULK it is possible to define global functions outside the scope of all classes. It is also possible to define a *single global expression* that constitutes the entry point to the program.

Most of the syntactic constructions in HULK are expressions, including conditional instructions and cycles. HULK is a statically typed language with optional type inference, which means that some (or all) parts of a program can be annotated with types, and the compiler will verify the consistency of all operations.

A.1.1 A didactic language

The HULK language has been designed as a mechanism for learning and evaluating a college course about compilers. For this reason, certain language design decisions respond more to didactic questions than to theoretical or pragmatic questions. An illustrative example is the inclusion of a single basic numerical type. In practice, programming languages have several numeric types (`int`, `float`, `double`, `decimal`) to cover the wide range of trade-off between efficiency and expressivity. However, from the didactic point of view, it is enough complexity

to have to deal with a numerical type, and the inclusion of others does not bring anything new from our point of view.

Another important decision is the static typing with type inference, which will be explained later in detail. The motivation behind this feature is to allow students to first implement an evaluator for the language, and then worry about type verification. Likewise, the decision to have global expressions, global functions, and classes, responds to the need to introduce the various elements of language little by little. By having global expressions, it is possible to implement an expression interpreter without the need to solve context-sensitive problems. Later, students can implement functions and finally the object-oriented features. In this way students can learn on the fly as they add characteristics to the language, always having a valid subset of the language implemented.

A.1.2 An incremental language

As its name indicates, HULK is a huge language. Actually, the HULK language really is not really a single programming language, but a set of programming languages. That is, HULK is designed as a set of layers, each with a new language feature that add increasingly more complex functionalities on top of the previous layers. It starts with a basic syntax for expressions, then global functions, and then a unified type system with simple inheritance. Afterwards, HULK grows to contain arrays, delegates, type inference, iterators, among other characteristics. All these language features have been designed to be compatible with each other. Furthermore, each language feature clearly describes on which other language features it depends.

This design has been conceived to allow the use of HULK at a wide range of learning levels. As a language of expressions and functions, it is useful for introductory courses on parsing and basic compilation techniques. Object orientation introduces a whole universe of semantic complexities; however, the HULK type system is simple enough to illustrate the most common problems in semantic type verification. Vectors introduce problems related to memory management, while anonymous functions and iterators are fundamentally problems of transpilation and code generation. The inference of types and the verification of null-safety is an exercise in logical inference, which can be used in advanced courses. The idea is that each course defines its objectives of interest, and can use an appropriate subset of HULK to illustrate and evaluate them.

A.1.3 BANNER: Intermediate Representation

Even though HULK can be defined without specific compilation details, we also provide a didactic 3-address code for intermediate representation that is convenient to use with HULK. For obvious reasons, it's called BANNER – **B**asic **3**-**A**dress **li**Near **i**Nt**E**mediate **R**epresentation.

A.2 Expressions

HULK is ultimately an expression-based language. Most of the syntactic constructions in HULK are expressions, including the body of all functions, loops, and any other block of code.

The body of a program in HULK always ends with a single global expression (and, if necessary, a final semicolon) that serves as the entrypoint of the program. This means that, of course, a program in HULK can consist of just one global expression.

For example, the following is a valid program in HULK:

```
42;
```

Obviously, this program has no side effects. A slightly more complicated program, probably the first one that does something, is this:

```
print(42);
```

In this program, `print` refers to a builtin function that prints the result of any expression in the output stream. We will talk about functions in a later section.

The rest of this section explains the basic expressions in HULK.

A.2.1 Arithmetic expressions

HULK defines three types of literal values: **numbers**, **strings**, and **booleans**. We will leave strings and booleans for later.

Numbers are 32-bit floating-point and support all basic arithmetic operations with the usual semantics: `+` (addition), `-` (subtraction), `*` (multiplication), `'(floating-point division)'`, `^` (power), and parenthesized sub-expressions.

The following is a valid HULK program that computes and prints the result of a rather useless arithmetic expression:

```
print((((1 + 2) ^ 3) * 4) / 5);
```

All usual syntactic and precedence rules apply.

A.2.2 Strings

String literals in HULK are defined within enclosed double-quotes (`"`), such as in:

```
print("Hello World");
```

A double-quote can be included literally by escaping it:

```
print("The message is \"Hello World\"");
```

Other escaped characters are `\n` for line endings, and `\t` for tabs.

Strings can be concatenated with other strings (or the string representation of numbers) using the `@` operator:

```
print("The meaning of life is " @ 42);
```

A.2.3 Builtin math functions and constants

Besides `print`, HULK also provides some common mathematical operations encapsulated as builtin functions with their usual semantics. The list of builtin math functions is the following:

- `sqrt(<value>)` computes the square root of a value.
- `sin(<angle>)` computes the sine of an angle in radians.
- `cos(<angle>)` computes the cosine of an angle in radians.
- `exp(<value>)` computes the value of `e` raised to a value.
- `log(<base>, <value>)` computes the logarithm of a value in a given base.
- `rand()` returns a random uniform number between 0 and 1 (both inclusive).

Besides these functions, HULK also ships with two global constants: `PI` and `E` which represent the floating-point value of these mathematical constants.

As expected, functions can be nested in HULK (provided the use of types is consistent, but so far all we care about is functions from numbers to numbers, so we can forget about types until later on). Hence, the following is a valid HULK program.

```
print(sin(2 * PI) ^ 2 + cos(3 * PI / log(4, 64)));
```

More formally, function invocation is also an expression in HULK, so everywhere you expect an expression you can also put a call to builtin function, and you can freely mix arithmetic expressions and mathematical functions, as you would expect in any programming language.

A.2.4 Expression blocks

Anywhere an expression is allowed (or almost), you can also use an expression block, which is nothing but a series of expressions between curly braces (`{` and `}`), and separated by `;`.

The most trivial usage of expression blocks is to allow multiple `print` statements as the body of a program. For example, the following is a valid HULK program:

```
{  
  print(42);  
  print(sin(PI/2));  
  print("Hello World");  
}
```

When you use an expression block instead of a single expression, it is often not necessary to end with a semicolon (`;`), but it is not erroneous to do so either.

A.3 Functions

HULK also lets you define your own functions (of course!). A program in HULK can have an arbitrary number of functions defined before the final global expression (or expression block).

A function's body is always an expression (or expression block), hence all functions have a return value (and type), that is, the return value (and type) of its body.

A.3.1 Inline functions

The easiest way to define a function is the inline form. Here's an example:

```
function tan(x) => sin(x) / cos(x);
```

An inline function is defined by an identifier followed by arguments between parenthesis, then the `=>` symbol, and then a simple expression (not an expression block) as body, ending in `;`.

In HULK, all functions must be defined before the final global expression. All these functions live in a single global namespace, hence it is not allowed to repeat function names. Similarly, there are no overloads in HULK (at least in “basic” HULK).

Finally, the body of any function can use other functions, regardless of whether they are defined before or after the corresponding function. Thus, the following is a valid HULK program:

```
function cot(x) => 1 / tan(x);  
function tan(x) => sin(x) / cos(x);  
  
print(tan(PI) ** 2 + cot(PI) ** 2);
```

And of course, inline functions (and any other type of function) can call themselves recursively.

A.3.2 Full-form functions

Since inline functions only allow for a single expression as body (as complex as that may be), HULK also allows full-form functions, in which the body is an expression block.

Here's an example of a rather useless function that prints 4 times:

```
function operate(x, y) {  
    print(x + y);  
    print(x - y);  
    print(x * y);  
    print(x / y);  
}
```

Note that the following form is discouraged for stylistic reasons:

```
function id(<args>) => {  
    // ...  
}
```

That is, you should either use the inline form with `=>` and a simple expression, or the full form with `{}` and an expression block.

A.4 Variables

Variables in HULK are lexically-scoped, which means that their scope is explicitly defined by the syntax. You use the `let` expression to introduce one or more variables and evaluate an expression in a new scope where those variables are defined.

The simplest form is introducing a single variable and using a single expression as body.

```
let msg = "Hello World" in print(msg);
```

Here `msg` is a new symbol that is defined *only* within the expression that goes after `in`.

A.4.1 Multiple variables

The `let` expression admits defining multiple variables at once like this:

```
let number = 42, text = "The meaning of life is" in  
  print(text @ number);
```

This is semantically equivalent to the following long form:

```
let number = 42 in  
  let text = "The meaning of life is" in  
    print(text @ number);
```

As you can notice, `let` associates to the right, so the previous is also equivalent to:

```
let number = 42 in (  
  let text = "The meaning of life is" in (  
    print(text @ number)  
  )  
);
```

A.4.2 Scoping rules

Since the binding is performed left-to-right (or equivalently starting from the outer `let`), and every variable is effectively bound in a new scope, you can safely use one variable when defining another:

```
let a = 6, b = a * 7 in print(b);
```

Which is equivalent to (and thus valid):

```
let a = 6 in  
  let b = a * 7 in  
    print(b);
```

A.4.3 Expression block body

You can also use an expression block as the body of a `let` expression:

```
let a = 5, b = 10, c = 20 in {  
  print(a+b);  
  print(b*c);  
  print(c/a);  
}
```

As we said before, semicolons (;) are seldom necessary after an expression block, but they are never wrong.

A.4.4 The `let` return value

As with almost everything in HULK, `let` is an expression, so it has a return value, which is obviously the return value of its body. This means the following is a valid HULK program:

```
let a = (let b = 6 in b * 7) in print(a);
```

Or more directly:

```
print(let b = 6 in b * 7);
```

This can be of course nested ad infinitum.

A.4.5 Redefining symbols

In HULK every new scope hides the symbols from the parent scope, which means you can redefine a variable name in an inner `let` expression:

```
let a = 20 in {  
  let a = 42 in print(a);  
  print(a);  
}
```

The previous code prints 42 then 20, since the inner `let` redefines the value of `a` inside its scope, but the value outside is still the one defined by the outer `let`.

And because of the [scoping rules](#), the following is also valid:


```
let a = 7, a = 7 * 6 in print(a);
```

Which is equivalent to:

```
let a = 7 in
  let a = 7 * 6 in
    print(a);
```

A.4.6 Destructive assignment

Most of the time in HULK you won't need to overwrite a variable, but there are cases where you do. In those cases, you can use the destructive assignment operator `:=`, like this:

```
let a = 0 in {
  print(a);
  a := 1;
  print(a);
}
```

The previous program prints 0 and then 1, since the value of `a` is overwritten before the second `print`. This is the **only** way in which a variable can be written to outside of a `let`.

As you would expect, the `:=` operator defines an expression too, which returns the value just assigned, so you can do the following:

```
let a = 0 in
  let b = a := 1 in {
    print(a);
    print(b);
  };
```

This is useful if you want to evaluate a complex expression to both test it (e.g, to see if its greater than zero) and store it for later use.

A.4.7 Rules for naming identifiers

Variables (and identifiers in general) in HULK can be named with any sequence of alphanumeric characters, plus underscore `_`, but must *always* begin with a letter (not a digit or `_`), hence the following are all valid identifiers:

- `x`
- `x0`
- `x_0`
- `lowercase`
- `TitleCase`
- `snake_case`
- `camelCase`

The following are invalid HULK identifiers:

- `_x`
- `x+y`
- `some method`
- `8ball`

And many others of course!

Since starting with an underscore `_` is invalid in user-produced HULK code, you will notice that when we talk about transpilation in HULK, variables and identifiers in transpiled code always start with `_`.

A.5 Conditionals

The `if` expression allows evaluating different expressions based on a condition.

```
let a = 42 in if (a % 2 == 0) print("Even") else print("odd");
```

Since `if` is itself an expression, returning the value of the branch that evaluated true, the previous program can be rewritten as follows:

```
let a = 42 in print(if (a % 2 == 0) "even" else "odd");
```

Conditions are just expressions of boolean type. The following are the valid boolean expressions:

- Boolean literals: `true` and `false`.
- Arithmetic comparison operators: `<`, `>`, `<=`, `>=`, `==`, `!=`, with their usual semantics.
- Boolean operators: `&` (and), `|` (or), and `!` (not) with their usual semantics.

A.5.1 Expression blocks in conditionals

The body of the `if` or the `else` part of a conditional (or both) can be an expression block as well:

```
let a = 42 in
  if (a % 2 == 0) {
    print(a);
    print("Even");
  }
  else print("Odd");
```

A.5.2 Multiple branches

The `if` expression supports multiple branches with the `elif` construction, which introduces another conditioned branch:

```
let a = 42, let mod = a % 3 in
  print(
    if (mod == 0) "Magic"
    elif (mod % 3 == 1) "Woke"
    else "Dumb"
  );
```

A.6 Loops

HULK defines two kinds of loops, the `while` expression and the `for` expression. Both loop constructions are expressions, returning the value of the

A.6.1 The while loop

A `while` loop evaluates a condition and its body while the condition is true. The body can be a simple expression or an expression block.

```
let a = 10 in while (a >= 0) {
  print(a);
  a := a - 1;
}
```

Since the return value of the `while` loop is the return value of its expression body, it can often be used directly as the body of a function.

```
function gcd(a, b) => while (a > 0)
  let m = a % b in {
    b := a;
    a := m;
  };
```

A.6.2 The for loop

A `for` loop iterates over an *iterable* of elements of a certain type. We will [talk about iterables](#) later on, but for now it suffices to say that if some expression evaluates to a collection, then the `for` loop can be used to iterate it.

For example, the builtin `range(<start>, <end>)` function evaluates to an iterable of numbers between `<start>` (inclusive) and `<end>` (non-inclusive).

```
for (x in range(0, 10)) print(x);
```

The `for` loop is semantically and operationally equivalent to the following:

```
let iterable = range(0, 10) in
  while (iterable.next())
    let x = iterable.current() in
      print(x);
```

In fact, what the reference implementation of the HULK compiler does in `for` loops is to transpile them into their `while` equivalent. This also effectively means that, just like the `while` loop, the `for` loop returns the last value of its body expression.

A.7 Types

HULK is ultimately an object-oriented language with simple inheritance and nominal typing. It also has features of structural typing via [protocols](#), which support language features such as [iterables](#), which we will explain later.

This section explains the basics of HULK's nominal typing system.

A type in HULK is basically a collection of attributes and methods, encapsulated under a type name. Attributes are always private, which means they can't be read or written to from any code outside the type in which they are defined (not even inheritors), while methods are always public and virtual.

A.7.1 Declaring types

A new type is declared using the `type` keyword followed by a name, and a body composed of attribute definitions and method definitions. All attributes must be given an initialization expression. Methods, like functions, can have a single expression or an expression block as body;

```
type Point {  
  x = 0;  
  y = 0;  
  
  getX() => self.x;  
  getY() => self.y;  
  
  setX(x) => self.x := x;  
  setY(y) => self.y := y;  
}
```

The body of every method is evaluated in a namespace that contains global symbols plus an especial symbol named `self` that references the current instance. The `self` symbol is **not** a keyword, which means it can be hidden by a `let` expression, or by a method argument.

However, when referring to the current instance, `self` is not a valid assignment target, so the following code should fail with a semantic error:

```
type A {  
  // ...  
  f() {  
    self := new A(); // <-- Semantic error, `self` is not a valid assignment target  
  }  
}
```

A.7.2 Instantiating types

To instantiate a type you use the keyword `new` followed by the type name:

```
let pt = new Point() in  
  print("x: " @ pt.getX() @ "; y: " @ pt.getY());
```

As you can see, type members are accessed by dot notation (`instance.member`).

You can pass arguments to a type, that you can use in the initialization expressions. This achieves an effect similar to having a single constructor.

```

type Point(x, y) {
  x = x;
  y = y;

  // ...
}

```

Then, at instantiation time, you can pass specific values:

```

let pt = new Point(3,4) in
  print("x: " @ pt.getX() @ "; y: " @ pt.getY());

```

Each attribute initialization expression is evaluated in a namespace that contains the global symbols and the type arguments, but not the `self` symbol. This means you cannot use other attributes of the same instance in an attribute initialization expression. This also means that you cannot assume any specific order of initialization of attributes.

A.7.3 Inheritance

Types in HULK can inherit from other types. The base of the type hierarchy is a type named `Object` which has no public members, which is the type you implicitly inherit from by default. To inherit from a specific type, you use the `inherits` keyword followed by the type name:

```

type PolarPoint inherits Point {
  rho() => sqrt(self.getX() ^ 2 + self.getY() ^ 2);
  // ...
}

```

By default, a type inherits its parent type arguments, which means that to construct a `PolarPoint` you have to pass the `x` and `y` that `Point` is expecting:

```

let pt = new PolarPoint(3,4) in
  print("rho: " @ pt.rho());

```

If you want to define a different set of type arguments, then you have to provide initialization expressions for the parent type at the declaration:

```

type PolarPoint(phi, rho) inherits Point(rho * sin(phi), rho * cos(phi)) {
  // ...
}

```

During construction, the expressions for type arguments of the parent are evaluated in a namespace that contains global symbols plus the type arguments of the inheritor. Like before, you cannot assume a specific order of evaluation.

In HULK, the three builtin types (`Number`, `String`, and `Boolean`) implicitly inherit from `Object`, but it is a semantic error to inherit from these types.

A.7.4 Polymorphism

All type methods in HULK are virtual by definition, and can be redefined by an inheritor provided the exact same signature is used:

```
type Person(firstname, lastname) {
  firstname = firstname;
  lastname = lastname;

  name() => self.firstname @@ self.lastname;
}
```

NOTE: `@@` is equivalent to `@ " " @`. It is a shorthand to insert a whitespace between two concatenated strings. There is no `@@@` or beyond, we're not savages.

```
type Knight inherits Person {
  name() => "Sir" @@ base();
}

let p = new Knight("Phil", "Collins") in
  print(p.name()); // prints 'Sir Phil Collins'
```

The `base` symbol in every method refers to the implementation of the parent (or the closest ancestor that has an implementation).

A.8 Type checking

HULK is a statically-typed language with optional type annotations. So far you haven't seen any because HULK has a powerful **type inference system** which we will talk about later on. However, all symbols in HULK have a static type, and all programs in HULK are statically checked during compilation.

Type annotations can be added anywhere a symbol is defined, that is:

- in variable declarations with `let` expressions;

- in function or method arguments and return type;
- in type attributes; and,
- in type arguments.

Let's see an example of each case.

A.8.1 Typing variables

Variables can be explicitly type-annotated in `let` expressions with the following syntax:

```
let x: Number = 42 in print(x);
```

The type checker will verify that the type inferred for the initialization expression is compatible with (formally, *conforms to*) the annotated type.

A.8.2 Typing functions and methods

All or a subset of a function's or method's arguments, and its return value, can be type-annotated with a similar syntax:

```
function tan(x: Number): Number => sin(x) / cos(x);
```

On the declaration side, the type checker will verify that the body of the method uses the types in a way that is consistent with their declaration. The exact meaning of this consistency is defined in the section about *type semantics*. The type checker will also verify that the return type of the body conforms to the annotated return type.

On the invocation side, the type checker will verify that the values passed as parameters conform to the annotated types.

Inside methods of a type `T`, the implicitly defined `self` symbol is always assumed as if annotated with type `T`.

A.8.3 Typing attributes and type arguments

In type definitions, attributes and type arguments can be type-annotated as follows:


```

type Point(x: Number, y: Number) {
  x: Number = x;
  y: Number = y;

  // ...
}

```

The type checker will verify that type arguments are used consistently inside attribute initialization expressions, and that the inferred type for each attribute initialization expression conforms to the attribute annotation.

A.8.4 Type conforming

The basic type relation in HULK is called *conforming* ($\leq$). A type T_1 is said to *conform to* another type T_2 (written as $T_1 \leq T_2$) if a variable of type T_2 can hold a value of type T_1 such that every possible operation that is semantically valid with T_2 is guaranteed to be semantically valid with T_1 .

In general, this means that the type checker will verify that the inferred type for any expression conforms to the corresponding type declared for that expression (e.g., the type of a variable, or the return type of a function).

The following rules provide an initial definition for the *conforming* relationship. The formal definition is given in the section about [type semantics](#).

- Every type conforms to `Object`.
- Every type conforms to itself.
- If T_1 inherits T_2 then T_1 conforms to T_2 .
- If T_1 conforms to T_2 and T_2 conforms to T_3 then T_1 conforms to T_3 .
- The only types that conform to `Number`, `String`, and `Boolean`, are respectively those same types.

Types in HULK form a single hierarchy rooted at `Object`. In this hierarchy the *conforming* relationship is equivalent to the *descendant* relationship. Thus, if T_1 conforms to T_2 that means that T_1 is a descendant of T_2 (or trivially the same type). Thus, we can talk of the lowest common ancestor of a set of types $T_1, T_2, \dots, T_n$, which is the most specific type T such that all T_i conform to T . When two types are in different branches of the type hierarchy, they are effectively incomparable.

NOTE: this conforming relationship is extended when we add [protocols](#).

A.8.5 Testing for dynamic types

The `is` operator allows to test an object to check whether its dynamic type conforms to a specific static type.

```
type Bird {
    // ...
}

type Plane {
    // ...
}

type Superman {
    // ...
}

let x = new Superman() in
    print(
        if (x is Bird) "It's bird!"
        elif (x is Plane) "It's a plane!"
        else "No, it's Superman!"
    );
```

In general, before the `is` operator you can put any expression, not just a variable.

A.8.6 Downcasting

You can use the `as` operator to downcast an expression to a given static type. The result is a runtime error if the expression is not a suitable dynamic type, which means you should always test if you're unsure:

```
type A {
    // ...
}

type B inherits A {
    // ...
}

type C inherits A {
```